

Semantic Endoscopic Image Segmentation Using Machine Learning

Your Name

June 4, 2025

1 Purpose of the Project

This project aims to systematically explore and benchmark modern semantic segmentation architectures on the CholecSeg8K dataset. Our goals are:

- Train and evaluate state-of-the-art segmentation models (e.g. DeepLabV3, SegFormer) to establish an upper performance bound.
- Develop a simple, custom U-Net model of comparable size and train it using standard techniques to obtain a baseline (lower bound) score.
- Incrementally refine the baseline model by incorporating advanced architectural modules and training strategies to narrow, or surpass, the gap between baseline and state-of-the-art performance.
- Provide a clear, reproducible PyTorch-based code framework for loading data, defining models, training with early stopping and hybrid losses, and conducting inference benchmarks.

2 Dataset: CholecSeg8K

CholecSeg8K is a publicly released dataset of annotated laparoscopic cholecystectomy frames, derived from the Cholec80 video corpus. Essential details:

- **Total images:** 8,080 frames extracted from 17 video clips.
- **Resolution:** original 854×480 px
- **Classes:** 13 semantic categories (e.g., abdominal wall, liver, cystic duct, grasper, blood).
- **Masks:** each frame has a hand-drawn label mask, a color visualization mask, and a watershed-ID mask.

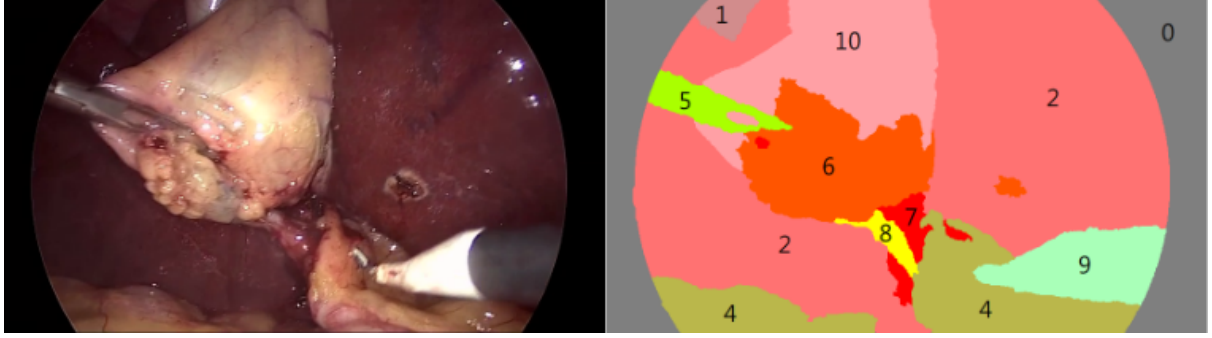


Figure 1: Original Image vs. Color Mask

2.1 Original Data Organization

The raw Cholec80 videos are structured as follows:

- `videoXX/` directories, one per surgery clip, each containing:
 - Consecutive frames extracted at 25 fps (e.g. `frame_0001_endo.png`)
 - Corresponding hand-drawn color masks (`frame_0001_endo_color_mask.png`)
 - Watershed-ID masks for annotation tool (`frame_0001_endo_mask.png`)
- Each subfolder spans 80 frames, resolution 854×480 , for 101 clips total.
- Final dataset: 8,080 frames with three mask types each.

2.2 Dataset Reorganization for Training

Before any modeling, we wrote a small Python script to flatten the original “videoXX/” folders into the standard ML layout:

- `dataset/images/`: all frames named `frame_x_endo.png`.
- `dataset/masks/`: matching masks named `frame_x_endo_color_mask.png`.
- This one-to-one naming and separate images/masks folders makes DataLoader logic trivial.

Note on Mask Types In an earlier, pre-experience run I only extracted the color masks (and converted them to grayscale during preprocessing), discarding the original watershed-ID masks.

- **Impact:** models train normally on these grayscale masks, but we lose the precise class-ID mapping for visualization.
- **Workaround:** since our focus is on error regions (where `pred != gt`), the lack of color-to-ID fidelity does not hinder our qualitative analysis.

3 Codebase Organization

To streamline training and evaluation across multiple architectures, we factor out common routines into shared modules.

3.1 Model Wrapper

The **ModelWrapper** class standardizes interaction with any segmentation model:

- Assigns a *name* and a *timestamp* so that each model's checkpoints, plots, and logs live in its own folder.
- Provides **predict()** to always return a (B, C, H, W) logits tensor, whether the model returns a dict (e.g. DeepLabV3) or a raw tensor.
- Implements **load_latest()** to find and load the most recent **.pth** file from the model's directory.
- Bundles utilities like **compute_iou()** and **denormalize()** for metrics and visualization.

Listing 1: ModelWrapper (simplified)

```
class ModelWrapper:
    def __init__(self, name, model):
        self.name      = name
        self.model      = model.to(DEVICE)
        self.timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    def predict(self, x):
        y = self.model(x)
        return y['out'] if isinstance(y, dict) else y

    def load_latest(self):
        files = glob.glob(f"{MODEL_PATH}/*.pth")
        latest = max(files, key=os.path.getctime)
        self.model.load_state_dict(torch.load(latest, map_location=DEVICE))
        return latest
```

3.2 Training Loop

The **train()** function follows this flow:

1. **Split** images/masks into 85% train and 15% validation.
2. Create **DataLoaders** with batching, shuffling (train only), and parallel I/O.
3. Initialize **AdamW** optimizer and **CrossEntropyLoss**.
4. **For each epoch:**
 - *Train phase:* call **predict()**, compute loss, backpropagate, and step optimizer.
 - *Validation phase:* forward on val set (no grads), compute loss and per-batch IoU.
 - *Checkpoint:* save best model on val loss improvement and every 10 epochs.
 - *Early stopping:* halt if val loss hasn't improved for *patience* epochs.

5. After training, plot **train/val loss** and **mIoU** curves over epochs.

Listing 2: Training Loop Outline

```
def train(model_wrapper, num_epochs, batch_size, lr, patience):
    1) data split & loaders
    2) model.to(DEVICE), optimizer, loss
    for epoch in range(1, num_epochs+1):
        train: model.train(), forward, loss.backward(), optimizer.step()
        validate: model.eval(), forward, compute IoU
        checkpoint & early stop
    plot metrics
```

3.3 Dataset Class

The **CholecSegDataset** handles image+mask pairing and preprocessing:

- *Transforms images*: Resize $\rightarrow$ ToTensor $\rightarrow$ Normalize (ImageNet mean/std).
- *Loads masks*: Convert to grayscale, Resize with **nearest-neighbor**, map 0–255 $\rightarrow$ 0...12 class IDs, wrap in a LongTensor.

Listing 3: CholecSegDataset

```
class CholecSegDataset(Dataset):
    def __init__(self, image_paths, mask_paths):
        self.img_transform = Compose([
            Resize((256,256)), ToTensor(),
            Normalize(mean, std)
        ])
    def __len__(self):
        return len(self.image_paths)
    def __getitem__(self, idx):
        img = self.img_transform(Image.open(...).convert('RGB'))
        mask = Image.open(...).convert('L')
        mask = mask.resize(..., NEAREST)
        mask = ((np.array(mask)/255)*(C-1)).round().astype('uint8')
        return img, torch.from_numpy(mask).long()
```

3.4 Metrics & Visualization

We track:

- **mIoU**: per-class IoU averaged over valid classes via `np.nanmean()`.
- **FPS**: inference throughput measured over a fixed number of samples.

For qualitative checks, we overlay an *error map* on the ground truth mask:

Listing 4: Error Overlay

```
mismatch = (pred != gt_mask)
ax.imshow(gt_mask, cmap='gray')
ax.imshow(mismatch, cmap='Reds', alpha=0.6)
```

4 State-of-the-Art Models

4.1 DeepLabV3

DeepLabV3 is a popular semantic segmentation model that combines a strong ImageNet-pretrained backbone with multi-scale context aggregation via Atrous Spatial Pyramid Pooling (ASPP).

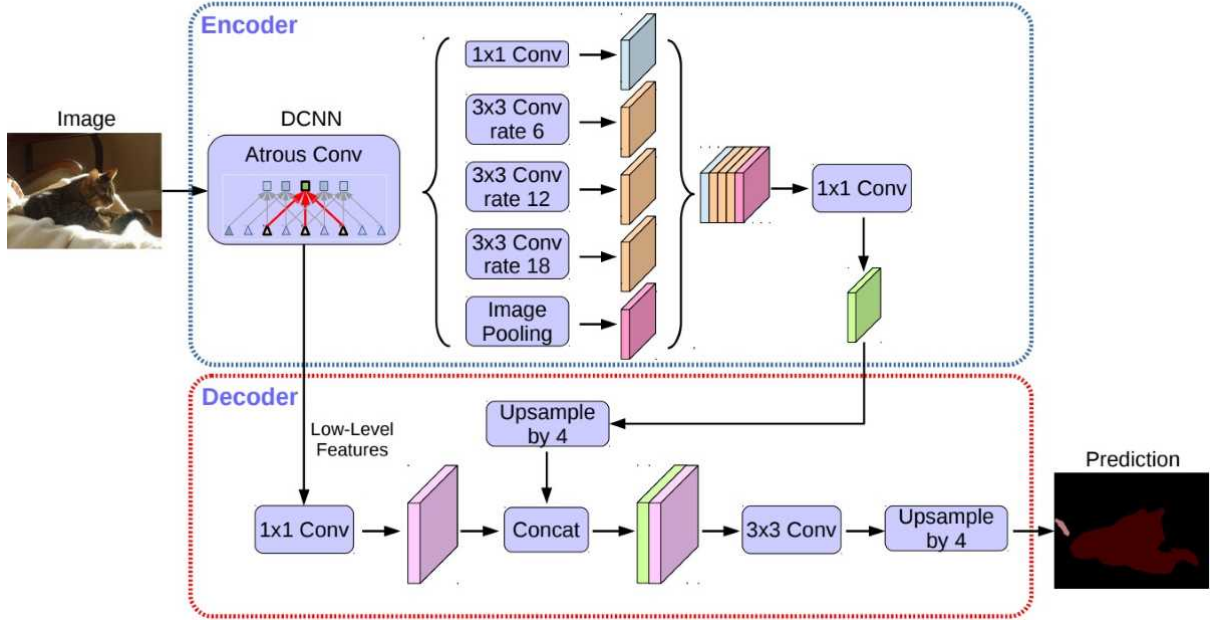


Figure 2: DeepLabV3 architecture: backbone $\rightarrow$ ASPP $\rightarrow$ decoder head.

Backbone & Atrous Convolutions

- Uses a ResNet-50 encoder, but replaces the last few 3×3 convolutions with *dilated* (atrous) convolutions.
- **Atrous convolution** expands the receptive field without reducing spatial resolution, preserving fine detail in feature maps.

ASPP (Atrous Spatial Pyramid Pooling) captures objects and context at multiple scales by running *four* parallel branches on the encoder's deepest feature map:

1. 1×1 convolution (no dilation)
2. 3×3 convolution with dilation rate = 6
3. 3×3 convolution with dilation rate = 12
4. 3×3 convolution with dilation rate = 18
5. Image-level pooling branch (global context)

The outputs of these five branches are concatenated and projected back to a fixed channel dimension via another 1×1 conv. This fused representation encodes both local detail and long-range context.

Decoder Head

- A lightweight decoder upsamples the ASPP output by $4\times$, concatenates it with a low-level feature map from early in the backbone (via a 1×1 conv projection), and applies a 3×3 conv.
- A final $4\times$ bilinear upsampling restores original image resolution, producing per-pixel logits.

Model Capacity & Implications

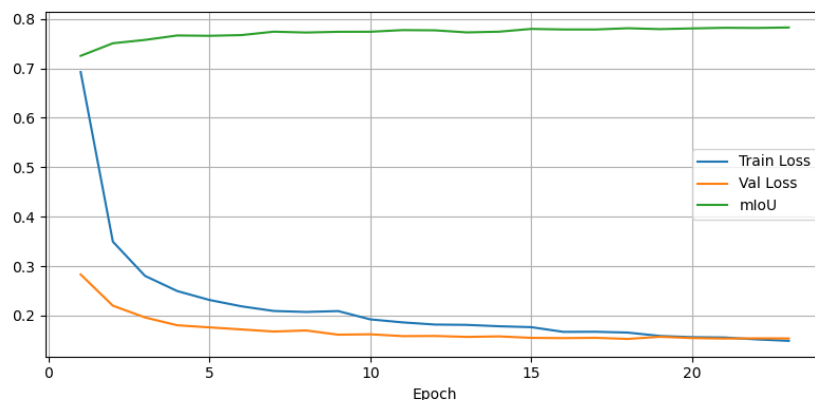
- Total parameters: approximately 40 million (25 M in ResNet-50 backbone + ASPP and decoder).
- Strengths: excellent multi-scale feature aggregation, strong baseline mIoU on VOC/Cityscapes.
- Considerations: heavier than plain U-Net; inference speed depends on dilation rates and ASPP complexity.

4.1.1 Training Configuration & Results

4.2 DeepLabV3: Training & Results

DeepLabV3 was trained using our unified script with the following hyperparameters:

- **Hyperparameters**
 - Max epochs: 23 (early stopping)
 - Batch size: 16 (32 can't fit on my GPUs due to high parameter count)
 - Optimizer: AdamW, learning rate = 1×10^{-4}
 - Loss: Cross-entropy
- **Training time:** 5841.7s total (254.0s/epoch)
- **Final mIoU:** 0.77
- **Inference speed:** 36 FPS



Discussion Although DeepLabV3 employs a large ResNet-50 backbone (25 M parameters) plus ASPP and a decoder (40 M total), its performance on CholecSeg8K was underwhelming compared to lighter models. Despite requiring over 1.6 hours to train (254 s/epoch) and delivering only 36 FPS at inference, it achieved a modest mIoU of 0.77. In contrast, smaller architectures (e.g., U-Net variants) often train faster and produce higher final mIoU. This suggests that simply scaling up parameter count and adding ASPP does not guarantee superior segmentation on endoscopic data—overparameterization can slow down both training and inference without proportional gains in accuracy.

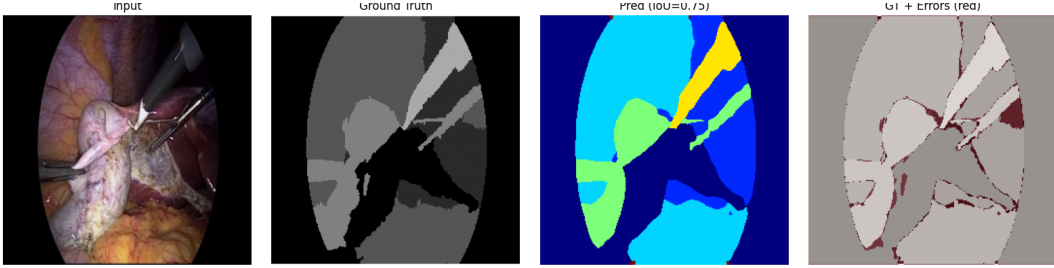


Figure 3: DeepLabV3 Inference Result

The inference result shown above illustrates a few notable failure modes, even though DeepLabV3 roughly segments the major structures. In particular:

- **Missed “islands” of small structures:** Tiny connected regions (e.g., thin vessels or small tool-tips) are often omitted or merged into the surrounding class rather than being correctly labeled as their own class.
- **Boundary fuzziness:** At the edges of organs—especially where two similarly-valued classes meet—the prediction sometimes “bleeds” or “rounds off” narrow protrusions, causing isolated pixels or narrow strips to vanish.

4.3 SegFormer-B0

SegFormer-B0 is a lightweight Transformer-based segmentation model with two key innovations:

- **MiT-B0 Encoder (Mix Transformer):**
 - *Overlapping patch embeddings:* each stage begins with a small Conv2D (kernel K_i , stride S_i , padding P_i) instead of non-overlapping patches, preserving locality.
 - *Hierarchical features:* produces four feature maps at resolutions $\frac{H}{4} \times \frac{W}{4}$, $\frac{H}{8} \times \frac{W}{8}$, $\frac{H}{16} \times \frac{W}{16}$, and $\frac{H}{32} \times \frac{W}{32}$ with channels [32, 64, 160, 256].
 - *Positional-encoding-free:* avoids fixed positional embeddings, making it robust to varied inference resolutions.
- **All-MLP Decoder:**
 - Each encoder feature map is projected through a 1×1 MLP to a common channel dimension, then upsampled to $\frac{H}{4} \times \frac{W}{4}$.

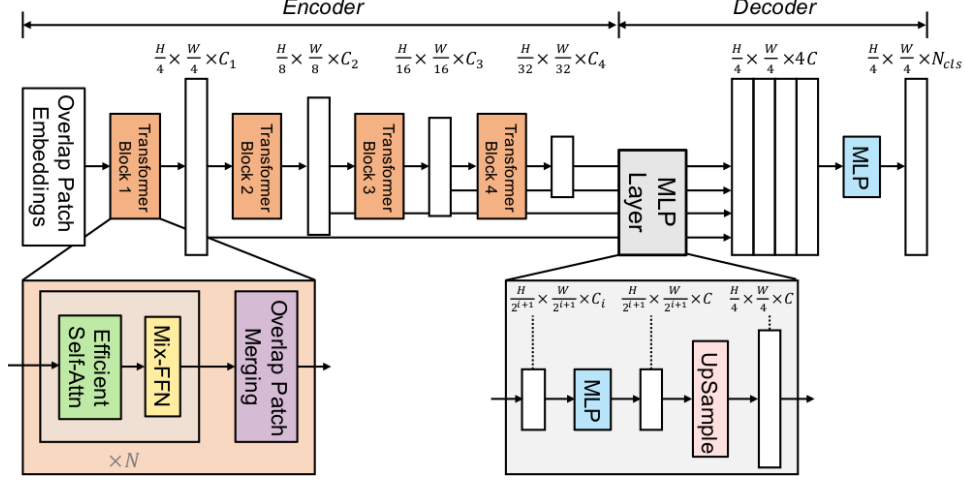


Figure 4: Segformer-B0 Architecture

- The four upsampled features are concatenated and passed through a final MLP to yield per-pixel logits for all classes.
- **Efficiency & Performance:**
 - Only ~ 3.8 million parameters in B0, enabling fast inference on edge devices.
 - Achieves competitive mIoU on ADE20K and Cityscapes despite its compact size.

4.3.1 Training Configuration & Results

- **Hyperparameters**
 - Max epochs: 40 (early stopping after 5 non-improving epochs)
 - Batch size: 32
 - Optimizer: AdamW, learning rate = 1×10^{-4}
 - Loss: Cross-entropy
- **Early stopping** triggered at epoch 35
- **Training time:** 1505.7 s total (43 s/epoch)
- **Final mIoU:** 0.9
- **Inference speed:** 39 FPS

4.3.2 Visualization

- Inference examples show that SegFormer-B0 not only segments large organs reliably but also recovers tiny, isolated clusters of secondary structures (e.g. small vessels, connective tissue edges) that other models often miss or smooth over.
- These fine-grained predictions appear as coherent, correctly classified pixel groups in the *Pred* panel, and their sparse errors (red overlay) confirm that the model truly learned boundary details rather than overfitting to broad regions.

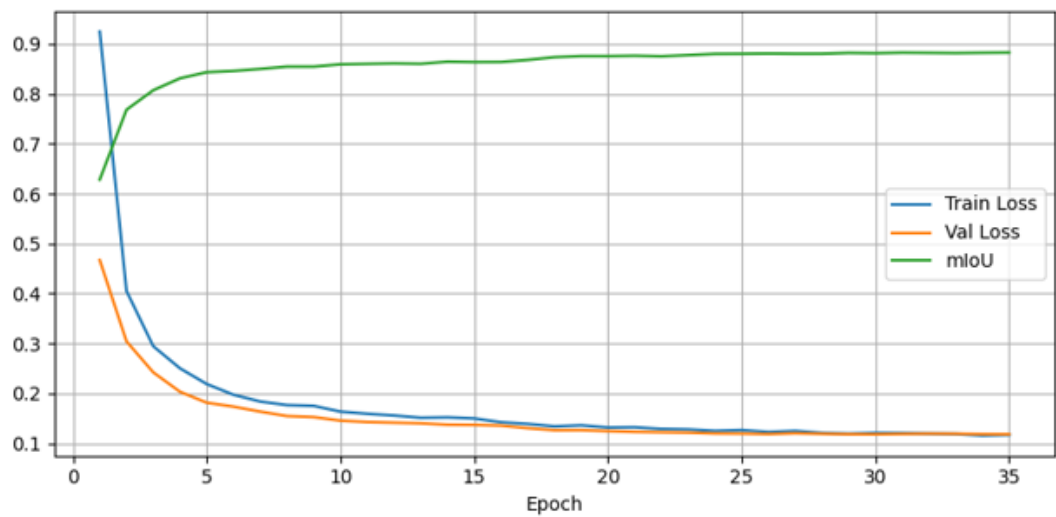


Figure 5: Train/Validation loss and mIoU curves for SegFormer-B0.

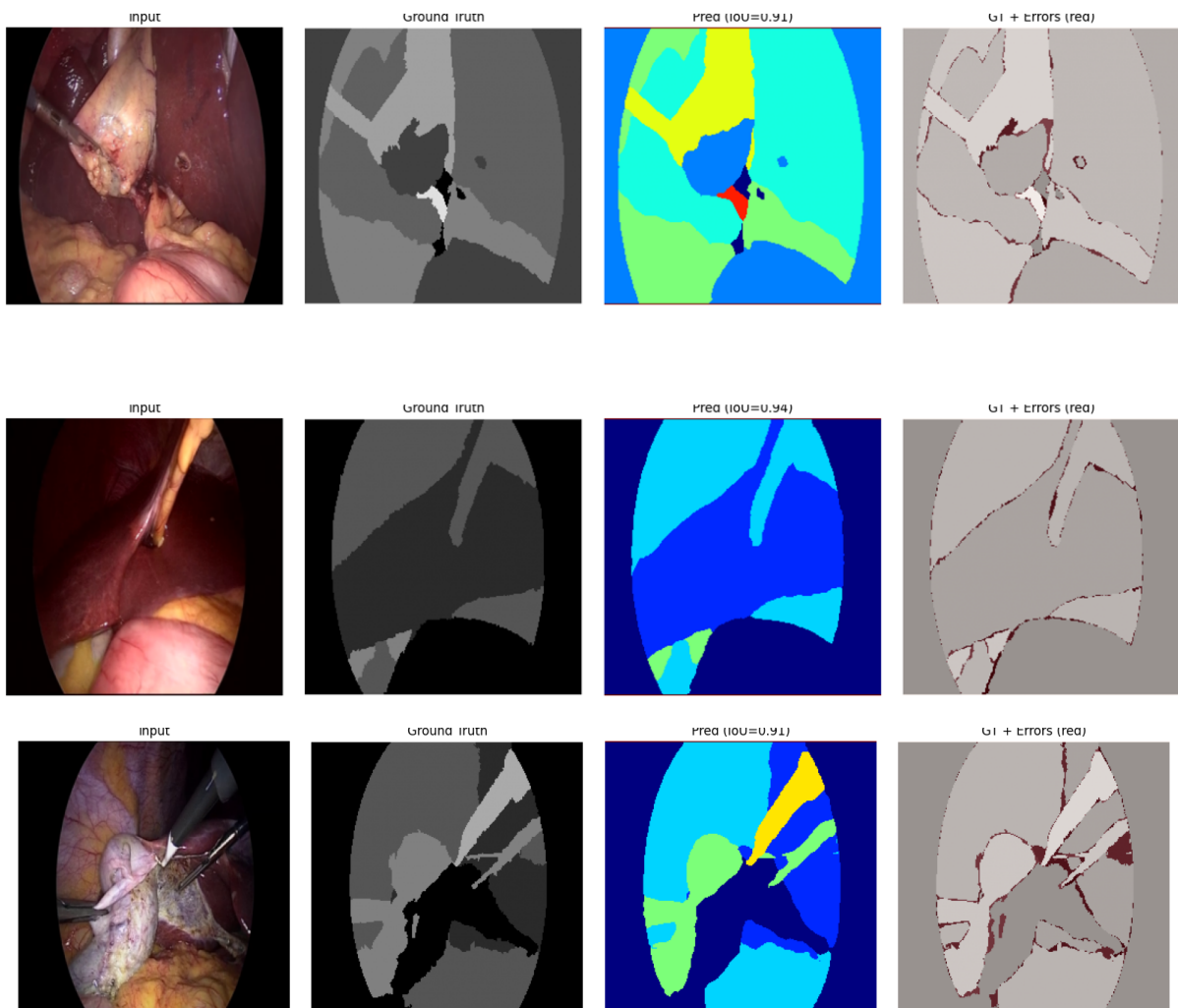


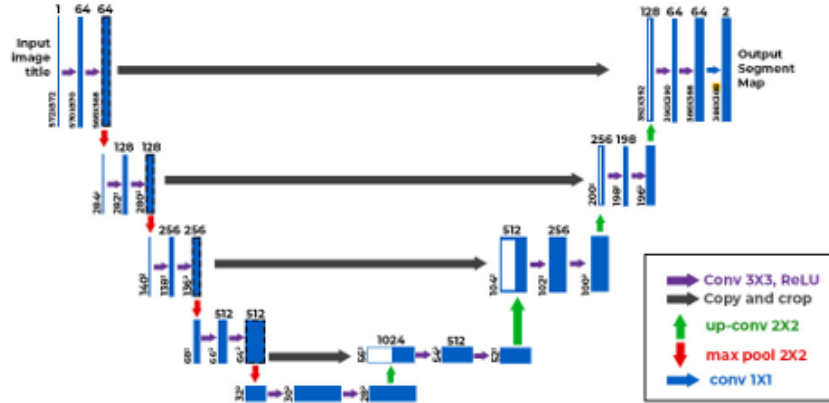
Figure 6: SegFormerB0 Inference Visualization

4.3.3 Remaining Limitations: Boundary Precision

- **Softened Edges:** Even with strong multi-scale context, SegFormer-B0 sometimes produces slightly blurred boundaries where two tissues meet (e.g. liver–fat interface), leading to mixed-class pixels along object margins.
- **Thin Structure Gaps:** Very narrow candida-like vessels or filmy connective strands can still be under-segmented or broken, especially when contrast against background is low.
- **Misaligned Contours:** Small spatial shifts (1–2 pixels) along curved edges occasionally occur, visible in the red error overlay as thin outlines around organs.
- **Interpolation Artifacts:** The final upsampling step (bilinear) can introduce minor aliasing along diagonal boundaries.

4.4 Basic U-Net (Baseline)

The foundational model in our experiments is a standard U-Net, inspired by the classical encoder–decoder structure shown below. This architecture serves as our *baseline* and consists of:



- **Encoder:** Four down-sampling levels, each containing two 3×3 convolutions with ReLU activations followed by a 2×2 max-pool. Channels grow from $3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$.
- **Decoder:** Three up-sampling steps via 2×2 transposed convolutions. At each level, the decoder merges (crop + concatenate) the corresponding encoder feature map and applies two more 3×3 convolutions + ReLU. Channels shrink from $256 \rightarrow 128 \rightarrow 64 \rightarrow 32$.
- **Segmentation Head:** A final 1×1 convolution maps the decoder's 32-channel output back to NUM_CLASSES per-pixel logits, followed by an optional bilinear up-sample to restore the original (H, W) .

This basic U-Net provides a concise yet powerful baseline for semantic segmentation on our dataset, allowing us to compare later enhancements (ResNet encoder, ASPP, attention, SE blocks, etc.) against a simple, widely used architecture.

Listing 5: Basic U-Net Implementation

```

class Block(Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.conv1 = Conv2d(in_channels, out_channels, kernel_size=3, padding=1)
        self.relu = ReLU(inplace=True)
        self.conv2 = Conv2d(out_channels, out_channels, kernel_size=3, padding=1)

    def forward(self, x):
        x = self.relu(self.conv1(x))
        x = self.relu(self.conv2(x))
        return x

class Encoder(Module):
    def __init__(self, channels=(3, 32, 64, 128, 256)):
        super().__init__()
        self.enc_blocks = ModuleList([
            Block(channels[i], channels[i+1]) for i in range(len(channels)-1)
        ])
        self.pool = MaxPool2d(kernel_size=2)

    def forward(self, x):
        features = []
        for block in self.enc_blocks:
            x = block(x)
            features.append(x)
            x = self.pool(x)
        return features

class Decoder(Module):
    def __init__(self, channels=(256, 128, 64, 32)):
        super().__init__()
        self.upconvs = ModuleList([
            ConvTranspose2d(channels[i], channels[i+1], kernel_size=2, stride=2)
            for i in range(len(channels)-1)
        ])
        self.dec_blocks = ModuleList([
            Block(channels[i+1]*2, channels[i+1]) for i in range(len(channels)-1)
        ])
        self.crop = CenterCrop

    def forward(self, x, encoder_features):
        for i in range(len(self.upconvs)):
            x = self.upconvs[i](x)
            enc_feat = encoder_features[-(i+1)]
            _, _, H, W = x.shape
            enc_cropped = self.crop([H, W])(enc_feat)
            x = torch.cat([x, enc_cropped], dim=1)
            x = self.dec_blocks[i](x)
        return x

```

```

class MyUNet(Module):
    def __init__(
        self,
        enc_channels=(3, 32, 64, 128, 256),
        dec_channels=(256, 128, 64, 32),
        num_classes=NUM_CLASSES,
        retain_dim=True,
        out_size=(IMAGE_HEIGHT, IMAGE_WIDTH)
    ):
        super().__init__()
        self.encoder = Encoder(enc_channels)
        self.decoder = Decoder(dec_channels)
        self.head = Conv2d(dec_channels[-1], num_classes, kernel_size=
        self.retain_dim = retain_dim
        self.out_size = out_size

    def forward(self, x):
        enc_feats = self.encoder(x)
        bottleneck = enc_feats[-1]
        dec_feats = self.decoder(bottleneck, enc_feats[:-1])
        logits = self.head(dec_feats)
        if self.retain_dim:
            logits = F.interpolate(logits, self.out_size,
                                   mode='bilinear', align_corners=False)

        return logits

```

4.4.1 Training & Results

- **Hyperparameters:**
 - Epochs: 50
 - Batch size: 32
 - Optimizer: AdamW, learning rate = 1×10^{-4}
 - Loss: Cross-entropy
- **Training Time:** 3079.6 s total (61.5 s/epoch)
- **Final mIoU:** 0.74
- **Inference Speed:** 50 FPS
- **Model Capacity:** 1,925,997 parameters

Discussion Despite its simplicity, the basic U-Net achieved an mIoU of 0.74 on the CholecSeg8K validation set. Its lightweight architecture (1.9 M parameters) also yields fast inference (50 FPS). However, there remain segmentation errors along fine object boundaries and small “islands,” motivating more advanced encoder and attention mechanisms in later models.

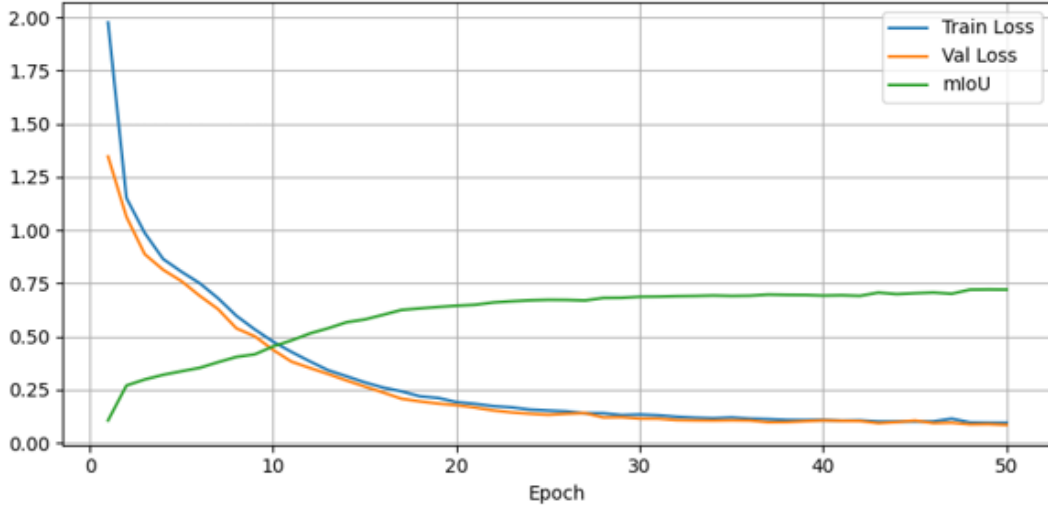


Figure 7: Train/Validation loss and mIoU curves for the basic U-Net.

4.5 Enhanced U-Net with Pretrained ResNet, ASPP, SE Blocks, and Attention Gates

The new model incrementally augments a basic U-Net by:

1. Replacing the plain convolutional encoder with a pretrained ResNet-34 encoder.
2. Inserting an Atrous Spatial Pyramid Pooling (ASPP) bottleneck for multi-scale context.
3. Adding Squeeze-and-Excitation (SE) channel attention within each decoder block.
4. Incorporating Attention Gates on skip connections to focus only on relevant features.

1. ResNet-34 Encoder (Pretrained) Instead of stacking simple “conv-ReLU → conv-ReLU → max-pool,” we reuse the first four stages of ResNet-34 (pretrained on ImageNet). These residual layers produce richer, multi-scale features:

Listing 6: ResNetEncoder

```
class ResNetEncoder(torch.nn.Module):
    def __init__(self, pretrained=True):
        super().__init__()
        rn = resnet34(pretrained=pretrained)
        self.initial = Sequential(
            rn.conv1, rn.bn1, rn.relu, rn.maxpool
        ) # f0: 64 channels, 1/4 spatial
        self.layer1 = rn.layer1 # f1: 64 channels, 1/4 spatial
        self.layer2 = rn.layer2 # f2:128 channels, 1/8 spatial
        self.layer3 = rn.layer3 # f3:256 channels, 1/16 spatial
        self.layer4 = rn.layer4 # f4:512 channels, 1/32 spatial

    def forward(self, x):
```

```

f0 = self.initial(x)
f1 = self.layer1(f0)
f2 = self.layer2(f1)
f3 = self.layer3(f2)
f4 = self.layer4(f3)
return [f1, f2, f3, f4]

```

- **Output:** A list $[f_1(64), f_2(128), f_3(256), f_4(512)]$. The deepest feature f_4 (512ch, 1/32 spatial) becomes input to ASPP; the others serve as skip connections.

2. ASPP Bottleneck (Multi-Scale Context) ASPP captures wide and narrow context by applying parallel atrous (dilated) convolutions on f_4 . Each branch sees a different effective receptive field:

Listing 7: ASPP Module

```

class ASPP(Module):
    """
    Atrous Spatial Pyramid Pooling for multi-scale context.
    """
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.conv1 = Conv2d(in_ch, out_ch, kernel_size=1)
        self.bn1 = BatchNorm2d(out_ch)
        self.conv6 = Conv2d(in_ch, out_ch, kernel_size=3,
                             padding=6, dilation=6)
        self.bn6 = BatchNorm2d(out_ch)
        self.conv12 = Conv2d(in_ch, out_ch, kernel_size=3,
                              padding=12, dilation=12)
        self.bn12 = BatchNorm2d(out_ch)
        self.conv18 = Conv2d(in_ch, out_ch, kernel_size=3,
                              padding=18, dilation=18)
        self.bn18 = BatchNorm2d(out_ch)
        self.project = Conv2d(out_ch*4, out_ch, kernel_size=1)
        self.bn_proj = BatchNorm2d(out_ch)
        self.relu = ReLU(inplace=True)

    def forward(self, x):
        y1 = self.relu(self.bn1(self.conv1(x)))
        y2 = self.relu(self.bn6(self.conv6(x)))
        y3 = self.relu(self.bn12(self.conv12(x)))
        y4 = self.relu(self.bn18(self.conv18(x)))
        y = torch.cat([y1, y2, y3, y4], dim=1) # cat 4 out_ch
        y = self.relu(self.bn_proj(self.project(y)))
        return y # shape: (B, out_ch, H/32, W/32)

```

- **Branches:**

- 1×1 convolution (no dilation)
- 3×3 atrous conv with rate=6, 12, and 18
- Concatenate then project back to out_ch via 1×1 conv

- **Benefit:**

- Large receptive field without further downsampling
- Fusion of local detail (dilation=6) and global context (dilation=18)

3. Decoder with SE Blocks and Attention Gates Each decoder stage upsamples, gates its corresponding skip, concatenates, then refines with two convs + SE.

Listing 8: SEBlock and AttentionGate

```
class SEBlock(Module):
    """
    Squeeze-and-Excitation for channelwise attention.
    """
    def __init__(self, channels, reduction=16):
        super().__init__()
        self.pool = AdaptiveAvgPool2d(1)
        self.fc1 = Linear(channels, channels//reduction, bias=False)
        self.fc2 = Linear(channels//reduction, channels, bias=False)
        self.relu = ReLU(inplace=True)
        self.sigmoid = Sigmoid()

    def forward(self, x):
        b, c, _, _ = x.size()
        y = self.pool(x).view(b, c) # global average (B, c)
        y = self.relu(self.fc1(y))
        y = self.sigmoid(self.fc2(y)).view(b, c, 1, 1)
        return x * y # channelwise reweighting

class AttentionGate(Module):
    """
    Spatial gating of skip features via coarse decoder context.
    """
    def __init__(self, F_g, F_l, F_int):
        super().__init__()
        self.W_g = Conv2d(F_g, F_int, kernel_size=1, bias=True)
        self.W_x = Conv2d(F_l, F_int, kernel_size=1, bias=True)
        self.psi = Conv2d(F_int, 1, kernel_size=1, bias=True)
        self.relu = ReLU(inplace=True)
        self.sigmoid = Sigmoid()

    def forward(self, g, x):
        # g: coarse decoder feature (B, F_g, H', W')
        # x: skip connection (B, F_l, H', W')
        g1 = self.W_g(g) # (B, F_int, H', W')
        x1 = self.W_x(x) # (B, F_int, H', W')
        psi = self.relu(g1 + x1)
        psi = self.sigmoid(self.psi(psi)) # (B, 1, H', W')
        return x * psi # gated skip
```

Listing 9: Decoder Block with SE

```
class Block(Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.conv1 = Conv2d(in_ch, out_ch, kernel_size=3, padding=1)
        self.bn1 = BatchNorm2d(out_ch)
        self.relu = ReLU(inplace=True)
        self.conv2 = Conv2d(out_ch, out_ch, kernel_size=3, padding=1)
        self.bn2 = BatchNorm2d(out_ch)
        self.se = SEBlock(out_ch)

    def forward(self, x):
        x = self.relu(self.bn1(self.conv1(x)))
        x = self.relu(self.bn2(self.conv2(x)))
        x = self.se(x) # channelwise recalibration
        return x
```

Listing 10: Decoder with Attention and SE

```
class Decoder(Module):
    def __init__(self, channels=(256, 256, 128, 64)):
        super().__init__()
        self.upconvs = ModuleList([
            ConvTranspose2d(channels[i], channels[i+1], kernel_size=2, stride=2)
            for i in range(len(channels)-1)
        ])
        self.attns = ModuleList([
            AttentionGate(F_g=channels[i+1], F_l=channels[i+1],
                          F_int=channels[i+1]//2)
            for i in range(len(channels)-1)
        ])
        self.dec_blocks = ModuleList([
            Block(channels[i+1]*2, channels[i+1])
            for i in range(len(channels)-1)
        ])
        self.crop = CenterCrop

    def forward(self, x, encoder_features):
        # encoder_features = [f1(64), f2(128), f3(256)]
        for i in range(len(self.upconvs)):
            x = self.upconvs[i](x) # upsample by 2
            skip = encoder_features[-(i+1)] # reverse: f 3 f2f1
            skip = self.attns[i](g=x, x=skip) # gate skip
            _, _, H, W = x.shape
            skip = self.crop([H, W])(skip) # center crop if needed
            x = torch.cat([x, skip], dim=1) # concat along channel axis
            x = self.dec_blocks[i](x) # two convs + SE
        return x
```

Listing 11: ModelProMaxUltra++ Full Architecture

```

class ModelProMaxUltra++(Module):
    def __init__(self, num_classes=NUM_CLASSES, out_size=(IMAGE_HEIGHT, IMAGE_WIDTH)):
        super().__init__()
        # 1) ResNet-34 encoder (pretrained)
        self.encoder = ResNetEncoder(pretrained=True)
        # 2) ASPP bottleneck on f4 (512, 256 channels)
        self.aspp = ASPP(in_ch=512, out_ch=256)
        # 3) Decoder with attention gates + SE blocks
        self.decoder = Decoder(channels=(256, 256, 128, 64))
        # 4) Final 1x1 conv to num_classes
        self.head = Conv2d(64, num_classes, kernel_size=1)
        self.out_size = out_size

    def forward(self, x):
        feats = self.encoder(x) # [f1(64), f2(128), f3(256), f4(512)]
        bottleneck = feats[-1] # f4: (B, 512, H/32, W/32)
        x = self.aspp(bottleneck) # (B, 256, H/32, W/32)
        x = self.decoder(x, feats[:-1]) # decode with [f3, f2, f1]
        logits = self.head(x) # (B, 64, H/4, W/4) or similar
        logits = F.interpolate(
            logits,
            self.out_size,
            mode='bilinear',
            align_corners=False
        ) # upsample to (H, W)
        return logits # (B, num_classes, H, W)

```

Why Each Component Matters

- **ResNet-34 Encoder:**

- Pretrained on ImageNet → better initialization, faster convergence.
- Residual blocks extract deeper semantic features than plain 3×3 conv stacks.

- **ASPP Bottleneck:**

- Parallel dilated convolutions (rates 6, 12, 18, plus 1×1) fuse multi-scale context.
- Maintains high spatial resolution (no pooling) for fine detail.

- **AttentionGate on Skip Connections:**

- Gating suppresses irrelevant skip activations (e.g. background) based on decoder context.
- Improves boundary localization and reduces false positives from distant regions.

- **SE Block (Squeeze-and-Excitation):**

- After two convs, channel-wise pooling + FC layers produce a per-channel weighting.
- Reweights feature channels to emphasize the most informative ones at each decoding stage.

4.6 Advanced Loss & Learning Rate Scheduling

In addition to the standard **cross-entropy** loss, we incorporate two key techniques to improve segmentation performance:

- **Dice Loss (Multiclass)**

- Measures overlap between the predicted mask and ground truth:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2 \sum_{c=1}^C \sum_i p_i^{(c)} g_i^{(c)}}{\sum_{c=1}^C \sum_i (p_i^{(c)})^2 + \sum_{c=1}^C \sum_i (g_i^{(c)})^2}$$

- *Advantages:*

- * Mitigates class imbalance by explicitly maximizing region overlap.
- * Encourages sharper boundaries and better small-object segmentation.

- In practice, we combine *Cross-Entropy* and *Dice* losses:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{Dice}}.$$

- **Cosine Annealing Learning Rate Schedule**

- We use `CosineAnnealingLR` with period $T_{\text{max}} = \text{num_epochs}$.
- Learning rate η_t follows:

$$\eta_t = \eta_{\text{min}} + \frac{1}{2}(\eta_{\text{max}} - \eta_{\text{min}}) \left(1 + \cos\left(\frac{t\pi}{T_{\text{max}}}\right) \right),$$

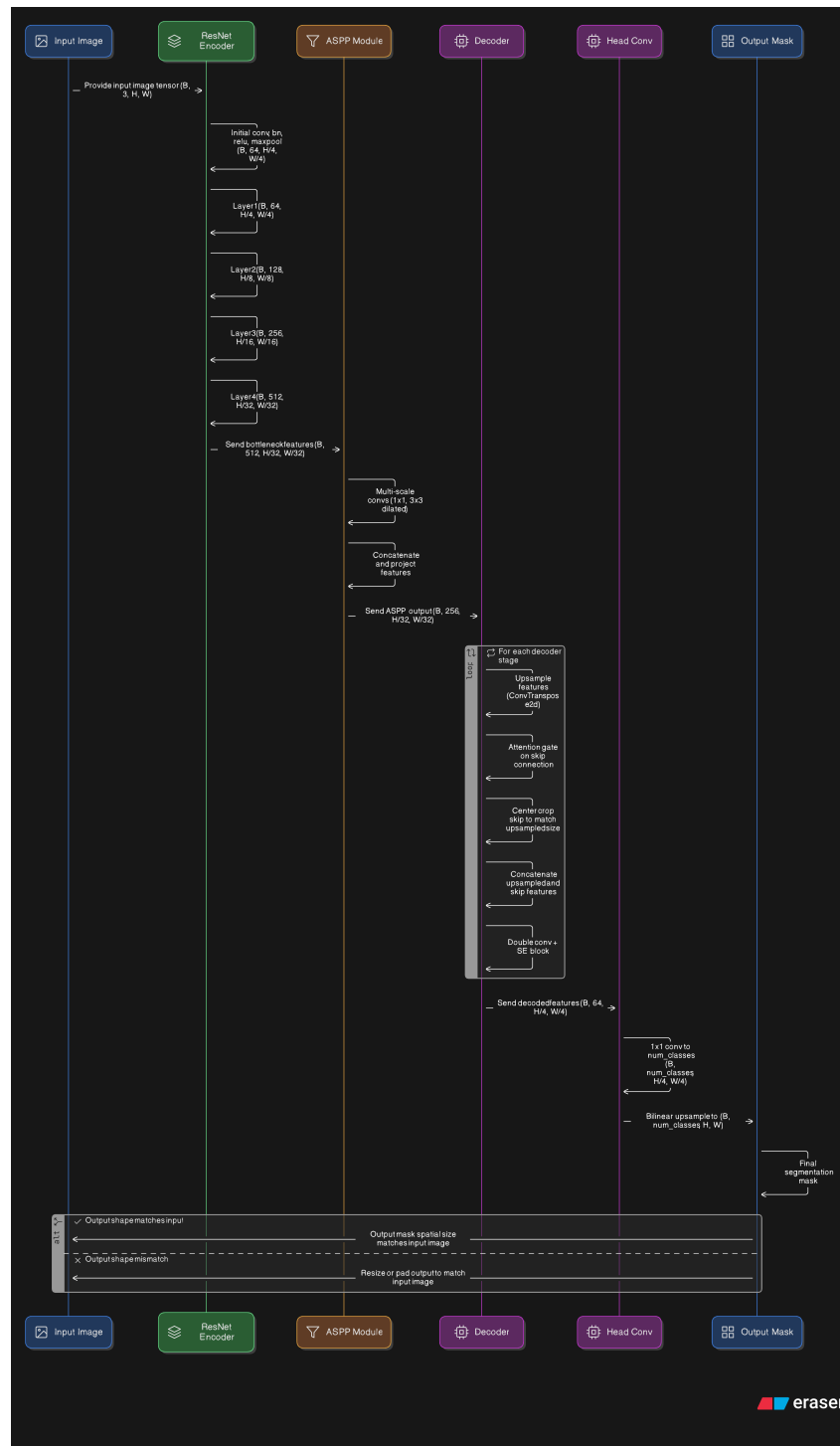
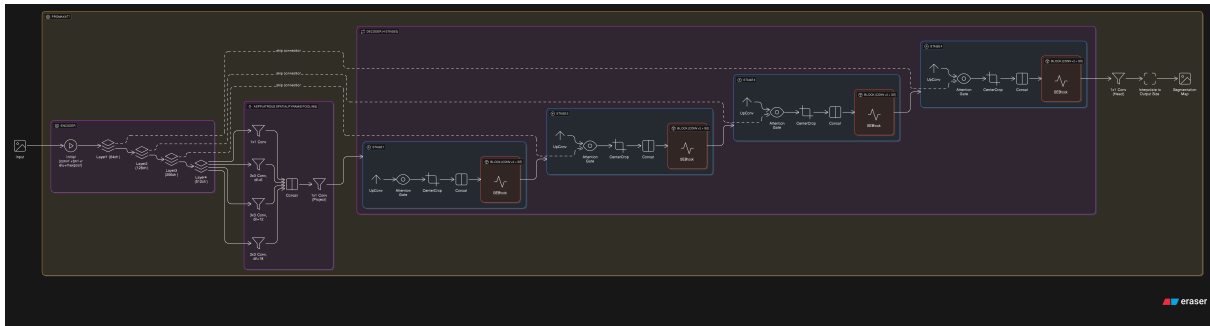
where η_{max} is the initial LR (1×10^{-4}) and $\eta_{\text{min}} = 0$.

- *Benefits:*

- * Smoothly decays the learning rate, avoiding abrupt drops.
- * Helps escape shallow minima early and refine weights later.

4.7 AI-Generated Architecture Diagrams

We experimented with generating architectural diagrams using an AI image tool. While the results appear somewhat “funny” stylistically, they nevertheless capture the overall structure of our model network with surprising fidelity. We include them here as a curiosity—both as a demonstration of current AI capabilities and as a fun, alternative visualization of our model.



4.8 Training Configuration & Results

- **Hyperparameters:**

- Epochs: 39 (early stopping triggered)
- Batch size: 32
- Optimizer: AdamW, learning rate = 1×10^{-4}
- Learning Rate Scheduler: CosineAnnealingLR with $T_{\max} = \text{num_epochs}$
- Loss: Cross-entropy + Dice

- **Training Time:** 1670.0 s total (42.8 s/epoch)

- **Final mIoU:** 0.86

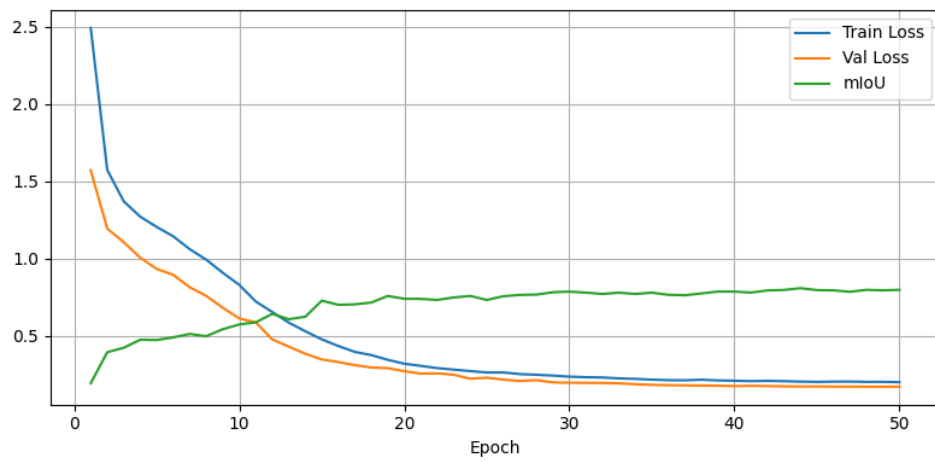
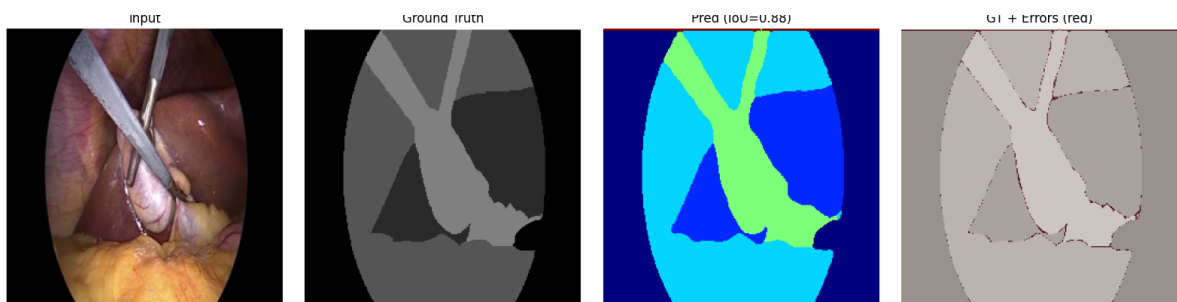


Figure 8: Train/Validation loss and mIoU curves for the final model.



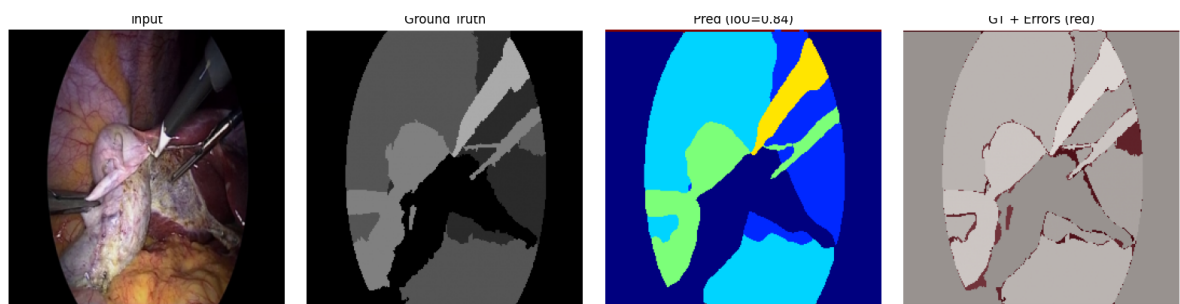


Figure 9: Inference Result